

Bubble Sort

Compare neighbours, swap inverted pairs, and lock the largest active value on the right after each pass.

This complete offline chapter connects ideas, diagrams, exact traces, commented Java and exam practice. Switch themes for reading; printing always uses the light presentation and includes every required trace and answer.

GUIDE 2026-10-07.1 · STANDALONE HTML AND PDF SHARE THE SAME CONTENT

1. The array we will sort

Arrays use zero-based positions. Sorting changes which values occupy those positions. The target is ascending order; record tags would move with their values.



Initial input [4,2,5,1,3]. Five fixed index positions 0..4.

We will distinguish comparisons, swaps, shifts, temporary writes and copy-back writes. They are different units.

2. Definition and plain language

Bubble sort is a comparison sort that repeatedly scans adjacent pairs. In ascending order it exchanges a pair only when the left value is larger than the right value.

Compare neighbours



Swap if inverted



Lock a rightmost value

The algorithm has a distinct decision and movement rule; each completed pass grows its known-sorted region.

Read the cards from left to right. Every time a bigger card sits before a smaller one, exchange them. A large card can travel many cells in one pass, while a small card may move left only one cell in that pass.



A concrete first pass: highlighted positions explain the choice; the green region records the completed result.

After one complete pass, the largest active value has reached the last active position. The next pass ignores that fixed suffix.

3. Vocabulary and the central invariant

Term	Meaning in this chapter
Adjacent	Two neighbouring indexes j and $j + 1$.
Swap	Exchange two values using a temporary variable.
Pass	One sweep through the still-active adjacent pairs.
Sorted suffix	The final positions already proved correct.
swapped flag	A Boolean reset before each pass; a whole pass with no change permits early termination.

Invariant: At the start of pass p (zero based), the last p positions hold the p largest values in final order.

4. Step through the complete execution

Predict the next decision before advancing. Next updates both the scene and the visibly marked trace row. Print includes the completed states and full table.

One comparison or write at a time

▼ Full trace table - select a row to jump to its scene

Step	Operation	Reason	Shared array
1	Pass 1, compare indexes 0 and 1	$4 > 2 \rightarrow \text{true}$; swap adjacent values; pass 1	[2, 4, 5, 1, 3]
2	Pass 1, compare indexes 1 and 2	$4 > 5 \rightarrow \text{false}$; leave the pair; pass 1	[2, 4, 5, 1, 3]

Step	Operation	Reason	Shared array
3	Pass 1, compare indexes 2 and 3	$5 > 1 \rightarrow \text{true}$; swap adjacent values; pass 1	[2, 4, 1, 5, 3]
4	Pass 1, compare indexes 3 and 4	$5 > 3 \rightarrow \text{true}$; swap adjacent values; pass 1	[2, 4, 1, 3, 5]
5	Pass 2, compare indexes 0 and 1	$2 > 4 \rightarrow \text{false}$; leave the pair; pass 2	[2, 4, 1, 3, 5]
6	Pass 2, compare indexes 1 and 2	$4 > 1 \rightarrow \text{true}$; swap adjacent values; pass 2	[2, 1, 4, 3, 5]
7	Pass 2, compare indexes 2 and 3	$4 > 3 \rightarrow \text{true}$; swap adjacent values; pass 2	[2, 1, 3, 4, 5]
8	Pass 3, compare indexes 0 and 1	$2 > 1 \rightarrow \text{true}$; swap adjacent values; pass 3	[1, 2, 3, 4, 5]
9	Pass 3, compare indexes 1 and 2	$2 > 3 \rightarrow \text{false}$; leave the pair; pass 3	[1, 2, 3, 4, 5]
10	Pass 4, compare indexes 0 and 1	$1 > 2 \rightarrow \text{false}$; leave the pair; pass 4	[1, 2, 3, 4, 5]

Pass 1, compare indexes 3 and 4

2

4

1

3

5

0

1

2

3

4

Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

$5 > 3 \rightarrow \text{true}$; swap adjacent values; pass 1

Pass 2, compare indexes 2 and 3

2

1

3

4

5

0

1

2

3

4

Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

$4 > 3 \rightarrow \text{true}$; swap adjacent values; pass 2

Pass 3, compare indexes 1 and 2

1

2

3

4

5

0

1

2

3

4

Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

$2 > 3 \rightarrow \text{false}$; leave the pair; pass 3

Pass 4, compare indexes 0 and 1



Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

1 > 2 → false; leave the pair; pass 4

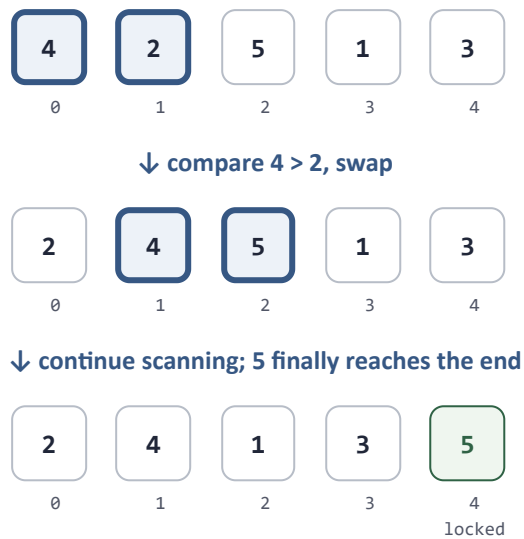
Final result: [1, 2, 3, 4, 5].

5. Exam-style completed-pass / return table

One row per outer-loop pass. These are the completed states to reproduce when an exam asks for the array after each pass.

Pass	Array before	Evaluated comparisons / decisions	Array after	Key comparisons	Swaps
1	[4,2,5,1,3]	4 > 2 → true; swap adjacent values; pass 1; 4 > 5 → false; leave the pair; pass 1; 5 > 1 → true; swap adjacent values; pass 1; 5 > 3 → true; swap adjacent values; pass 1	[2,4,1,3,5]	4	3
2	[2,4,1,3,5]	2 > 4 → false; leave the pair; pass 2; 4 > 1 → true; swap adjacent values; pass 2; 4 > 3 → true; swap adjacent values; pass 2	[2,1,3,4,5]	3	2
3	[2,1,3,4,5]	2 > 1 → true; swap adjacent values; pass 3; 2 > 3 → false; leave the pair; pass 3	[1,2,3,4,5]	2	1
4	[1,2,3,4,5]	1 > 2 → false; leave the pair; pass 4	[1,2,3,4,5]	1	0

6. What actually moves, and why



One swap uses two writes to input-array cells; the temporary variable assignment is not an array write.

One non-swap does not prove the whole array is sorted. Only a full no-swap pass supports the early stop.

7. Fully commented Java implementation

JAVA

```
public static void bubbleSort(int[] arr) {
    // After pass p, the largest p+1 values are fixed on the right.
    for (int pass = 0; pass < arr.length - 1; pass++) {
        boolean swapped = false; // Reset for THIS pass.
        // Skip the sorted suffix; j+1 remains a valid index.
        for (int j = 0; j < arr.length - 1 - pass; j++) {
            // Strict comparison leaves equal values in their original order.
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j]; // Save before overwriting.
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        // A whole pass with no swap proves every adjacent pair is ordered.
        if (!swapped) break;
    }
}
```

Write it from scratch

- Start a new pass and reset swapped.
- Compare neighbours within the active prefix.

- Swap strictly inverted pairs.
- Shorten the next pass by one fixed position.
- Stop after a whole no-swap pass.

8. Recursion, calls and returns

JAVA

```
public static void bubbleSortRec(int[] arr, int n) {
    // n is the active prefix LENGTH, not an index.
    if (n <= 1) return;
    boolean swapped = false;
    for (int j = 0; j < n - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
            swapped = true;
        }
    }
    // The largest active value is now at n-1; shrink the prefix.
    if (swapped) bubbleSortRec(arr, n - 1);
    // First call: bubbleSortRec(arr, arr.length).
}
```

CALL STACK - NEWEST FRAME AT THE TOP

bubbleSortRec(n=4)

Pass fixes the largest remaining value at index 3

bubbleSortRec(n=5)

Waiting: Child sorts remaining prefix 0..3

Unlike iterative auxiliary space, suspended recursive frames require $O(n)$ stack space in the worst case.

9. Complete runnable program

Save as BubbleDemo.java. The code includes the method(s) and a main entry point.

JAVA

```
import java.util.Arrays;

public class BubbleDemo {
    public static void bubbleSort(int[] arr) {
        // After pass p, the largest p+1 values are fixed on the right.
        for (int pass = 0; pass < arr.length - 1; pass++) {
            boolean swapped = false; // Reset for THIS pass.
            // Skip the sorted suffix; j+1 remains a valid index.
```

```

        for (int j = 0; j < arr.length - 1 - pass; j++) {
            // Strict comparison leaves equal values in their original order.
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j]; // Save before overwriting.
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        // A whole pass with no swap proves every adjacent pair is ordered.
        if (!swapped) break;
    }
}

public static void main(String[] args) {
    int[] values = {4, 2, 5, 1, 3};
    bubbleSort(values);
    System.out.println(Arrays.toString(values));
}
}

```

EXPECTED CONSOLE OUTPUT

[1, 2, 3, 4, 5]

10. Descending order: change the decision

Change `arr[j] > arr[j + 1]` to `arr[j] < arr[j + 1]`. Each pass then locks the smallest active value on the right.

Keep the bounds and movement rules consistent. Reversing a final array is a different procedure and does not reproduce the descending trace of this implementation.

11. Five worked examples

Each case has input and result diagrams, evaluated decisions or merge inputs, exact counts, and a final state.

1. Nearly sorted

1

2

4

3

5

0

1

2

3

4

↓ ascending bubble sort

1

2

3

4

5

0

1

2

3

4

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[1,2,4,3,5]	1 > 2 → false; leave the pair; pass 1; 2 > 4 → false; leave the pair; pass 1; 4 > 3 → true; swap adjacent values; pass 1; 4 > 5 → false; leave the pair; pass 1	[1,2,3,4,5]	4
2	[1,2,3,4,5]	1 > 2 → false; leave the pair; pass 2; 2 > 3 → false; leave the pair; pass 2; 3 > 4 → false; leave the pair; pass 2	[1,2,3,4,5]	3

7 key comparisons; 1 swaps.

2. Reverse order

5

4

3

2

1

0

1

2

3

4

↓ ascending bubble sort

1

2

3

4

5

0

1

2

3

4

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[5,4,3,2,1]	5 > 4 → true; swap adjacent values; pass 1; 5 > 3 → true; swap adjacent values; pass 1; 5 > 2 → true; swap adjacent values; pass 1; 5 > 1 → true; swap adjacent values; pass 1	[4,3,2,1,5]	4
2	[4,3,2,1,5]	4 > 3 → true; swap adjacent values; pass 2; 4 > 2 → true; swap adjacent values; pass 2; 4 > 1 → true; swap adjacent values; pass 2	[3,2,1,4,5]	3
3	[3,2,1,4,5]	3 > 2 → true; swap adjacent values; pass 3; 3 > 1 → true; swap adjacent values; pass 3	[2,1,3,4,5]	2
4	[2,1,3,4,5]	2 > 1 → true; swap adjacent values; pass 4	[1,2,3,4,5]	1

10 key comparisons; 10 swaps.

3. Duplicate values

2

2

1

3

0

1

2

3

↓ ascending bubble sort

1

2

2

3

0

1

2

3

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[2,2,1,3]	2 > 2 → false; leave the pair; pass 1; 2 > 1 → true; swap adjacent values; pass 1; 2 > 3 → false; leave the pair; pass 1	[2,1,2,3]	3
2	[2,1,2,3]	2 > 1 → true; swap adjacent values; pass 2; 2 > 2 → false; leave the pair; pass 2	[1,2,2,3]	2
3	[1,2,2,3]	1 > 2 → false; leave the pair; pass 3	[1,2,2,3]	1

6 key comparisons; 2 swaps.

4. Uneven length

38

27

43

3

9

0

1

2

3

4

↓ ascending bubble sort

3

9

27

38

43

0

1

2

3

4

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[38,27,43,3,9]	38 > 27 → true; swap adjacent values; pass 1; 38 > 43 → false; leave the pair; pass 1; 43 > 3 → true; swap adjacent values; pass 1; 43 > 9 → true; swap adjacent values; pass 1	[27,38,3,9,43]	4
2	[27,38,3,9,43]	27 > 38 → false; leave the pair; pass 2; 38 > 3 → true; swap adjacent values; pass 2; 38 > 9 → true; swap adjacent values; pass 2	[27,3,9,38,43]	3
3	[27,3,9,38,43]	27 > 3 → true; swap adjacent values; pass 3; 27 > 9 → true; swap adjacent values; pass 3	[3,9,27,38,43]	2
4	[3,9,27,38,43]	3 > 9 → false; leave the pair; pass 4	[3,9,27,38,43]	1

10 key comparisons; 7 swaps.

5. Descending target

8

3

5

1

0

1

2

3

↓ descending bubble sort

8

5

3

1

0

1

2

3

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[8,3,5,1]	$8 < 3 \rightarrow \text{false}$; leave the pair; pass 1; $3 < 5 \rightarrow \text{true}$; swap adjacent values; pass 1; $3 < 1 \rightarrow \text{false}$; leave the pair; pass 1	[8,5,3,1]	3
2	[8,5,3,1]	$8 < 5 \rightarrow \text{false}$; leave the pair; pass 2; $5 < 3 \rightarrow \text{false}$; leave the pair; pass 2	[8,5,3,1]	2

5 key comparisons; 1 swaps. Change $\text{arr}[j] > \text{arr}[j + 1]$ to $\text{arr}[j] < \text{arr}[j + 1]$. Each pass then locks the smallest active value on the right.

12. Properties and why the complexity occurs

Property	Result	Reason
Best	$O(n)$	Already sorted input: one pass makes $n - 1$ comparisons and no swaps.
Average / worst	$O(n^2)$	A reverse-order array needs many shrinking sweeps.
Auxiliary space	$O(1)$ iterative; $O(n)$ recursive worst case	A temporary swap value versus a chain of calls.
Stable / in place	Yes / yes	Strict $>$ avoids crossing equals; values are exchanged inside the input.
Adaptive	Yes in this optimized version	The swapped flag enables early stop.

Pass 1		$n - 1$ comparisons
Pass 2		$n - 2$ comparisons
Last pass		1 comparison

Worst-case triangular work: $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$.

On the main input there are 10 key comparisons and 6 swaps. A swap writes two input cells, giving 12 array writes. A no-swap first pass on a sorted five-value array uses only 4 comparisons.

13. When to choose it

Useful for small teaching examples and for recognizing adjacent-pair logic. Already sorted input benefits from early stop; large unsorted arrays are better suited to an $O(n \log n)$ algorithm.

14. Common mistakes, symptoms and repairs

Attempt	What goes wrong	Repair
Loop to $j < \text{arr.length}$	$\text{arr}[j + 1]$ reaches an invalid index.	Use $j < \text{arr.length} - 1$ - pass.
Declare swapped only once	An earlier swap prevents later early termination.	Reset it to false before every pass.
Swap on $\geq$	Equal records may cross and lose stability.	Swap only strictly inverted pairs.
Stop after one non-swap	Only that pair is ordered.	Stop after an entire pass with no swaps.

15. Exam-style practice and worked answers

A. Trace [8, 3, 5, 1] in descending order. Show every completed pass/merge. [8]

Pass	Before	After	Key comparisons
1	[8,3,5,1]	[8,5,3,1]	3
2	[8,5,3,1]	[8,5,3,1]	2

Final result [8,5,3,1]. Change $\text{arr}[j] > \text{arr}[j + 1]$ to $\text{arr}[j] < \text{arr}[j + 1]$. Each pass then locks the smallest active value on the right.

B. Explain stability with tagged equal keys. [4]

Strict inversion comparisons do not move a value past an equal one. Using $\geq$ (or $\leq$ descending) would remove that guarantee.

C. Give the time and space, then justify both. [5]

Best: $O(n)$. Already sorted input: one pass makes $n - 1$ comparisons and no swaps. Average / worst: $O(n^2)$. A reverse-order array needs many shrinking sweeps. Auxiliary space: $O(1)$ iterative; $O(n)$ recursive worst case. A temporary swap value versus a chain of calls. Stable / in place: Yes / yes. Strict $>$ avoids crossing equals; values are exchanged inside the input. Adaptive: Yes in this optimized version. The swapped flag enables early stop.

D. Debug the code before tracing it. [4]

Mistake	Symptom	Fix
Loop to <code>j < arr.length</code>	<code>arr[j + 1]</code> reaches an invalid index.	Use <code>j < arr.length - 1 - pass</code> .
Declare swapped only once	An earlier swap prevents later early termination.	Reset it to false before every pass.

E. What happens for [] and [7]? [2]

Both are already sorted. No key comparisons or value movements are required. Merge sort returns at its base case; the iterative methods never enter a pass.

16. Test-ready summary and mastery check

Compare neighbours, swap inverted pairs, and lock the largest active value on the right after each pass.

- I can introduce every variable before using it.
- I can draw the current array with index labels.
- I can evaluate conditions with the actual values.
- I can distinguish pass summaries from individual operations.
- I can trace descending and duplicate cases.
- I can justify stability, time and auxiliary space.
- I can test empty, singleton, sorted and reverse input.

CSI247 visual study guide · Bubble Sort · 2026-10-07.1

Structural reference: the author-supplied CSI247 guide, with topic-specific adaptation and repo-owned teaching content. All traces in this artifact are generated by deterministic simulations of its stated procedures. Counts distinguish key comparisons from loop tests, probes, swaps, shifts and array writes.